

Тема «Основные операторы JavaScript. Функции»

Содержание:

- 1 Основные операторы языка сценариев JavaScript
 - 1.1 Арифметические операторы
 - 1.2 Битовые операторы
 - 1.3 Операторы присваивания
 - 1.4 Логические операторы
 - 1.5 Строковые операторы
 - 1.6 Условные операторы
 - 1.7 Операторы цикла
 - 1.8 Операторы для работы с объектами
 - 1.9 Комментарии
- 2 Старшинство операций
- 3 Использование функций в скрипт-программах

1 Основные операторы языка сценариев JavaScript

В JavaScript имеется несколько типов выражений (и соответственно операторов):

- арифметические (результатом является числовое значение);
- строковые (результатом является строка, например, “Helen” или “567”);
- логические (вычисляют значения true или false).

Специальное ключевое слово null задает нулевое значение. Если переменной не было присвоено никакого значения, она является неопределенной (undefined) и применение ее в программе приведет к ошибке.

Операторы могут быть **бинарными** и **унарными**. В бинарных операциях присутствуют два операнда, один до, а другой после оператора:

operand1 operator operand2

Например, 3+4 или x*y.

Унарные операции содержат один операнд, до или после оператора:

Operator operand

Например, x++ или ++x.

С точки зрения реализации базовых структур операторы языка сценариев JavaScript могут быть линейными, условными и циклическими.

1.1 Арифметические операторы

Арифметические операторы используют числовые значения операндов, литералы либо переменные. Результатом выполнения будет единственное числовое значение. Стандартными арифметическими операциями являются сложение (+), вычитание (-), умножение (*) и деление (/).

Модуль числа (%) – остаток от деления.

Формат оператора:

Например, результатом операции 12 % 5 будет 2. **А наоборот?**

Инкрементация (++).

Формат оператора:

`var++` или `++var`

Операция добавляет единицу к значению операнда. Существуют две формы операции инкрементации: префиксная (`++var`) и суффиксная (`var++`). В префиксной форме происходит инкрементация операнда и возвращение результата операции. В суффиксной форме инкрементация операнда выполняется после возвращения результата операции.

Примеры:

```
x=3
y=x++
Значением y будет 3, а значением x – 4.
x=3
y=++x
Значением x и y будет 4.
```

Декрементация (--).

Формат оператора:

`var--` или `--var`

Операция вычитает единицу из значения операнда. Существуют две формы операции декрементации: префиксная (`--var`) и суффиксная (`var--`). В префиксной форме происходит декрементация операнда и возвращение результата операции. В суффиксной форме декрементация операнда выполняется после возвращения результата операции.

Примеры:

```
x=3
y=x--
Значением y будет 3, а значением x – 2.
x=3
y=--x
Значением x и y будет 2.
```

Отрицание (-).

Операция меняет знак на противоположный.

Пример:

```
y=-x
```

1.2 Битовые операторы

Битовые операторы рассматривают операнды как набор битов (значений 0 и 1), но возвращают результат в стандартном для JavaScript представлении числовых значений.

1.2.1 Логические битовые операции

Концептуально логические битовые операции работают следующим образом:

- операнды конвертируются в 32-битные целые числа;
- выполняются операции попарно для соответствующих битов обоих операндов (первый с первым, второй со вторым и так далее).

К логическим битовым операциям относят следующие:

& (AND) возвращает значение 1, если оба бита равны 1;

| (OR) возвращает значение 1, если хотя бы один бит равен 1;

^ (XOR) возвращает значение 1, если один из битов равен 1.

Ставит 1 на бит результата, для которого только один из соответствующих битов операндов равен 1 (но не оба).

Пример: двоичное представление 9 – 1001, а 15 – 1111.

15 & 9=9 (1111 & 1001=1001)

15 | 9=15 (1111 | 1001=1111)

15 ^ 9=6(1111 ^ 1001=0110)

1.2.1 Операторы сдвига

Операции сдвига требуют два операнда: первый – сдвигаемый операнд и второй – указатель сдвига (число сдвигаемых битов). Направление сдвига определяется выбранным знаком операции. Операции сдвига конвертируют значения операндов к 32-битному целому представлению, а результат сдвига возвращается в формате первого операнда.

Операция левого сдвига (<<).

Операция сдвигает биты первого операнда влево на число позиций, указанных во втором операнде. Значения выдвинутых битов пропадают, а на место сдвинутых битов справа записываются нули.

Например, $9 \ll 2 = 36$.

Операция правого сдвига с расширением знака (>>).

Операция сдвигает биты первого операнда вправо на число позиций, указанных во втором операнде. Значения выдвинутых битов пропадают, а на место сдвинутых битов слева записывается значение левого «знакового» бита операнда.

Например,

$9 \gg 2 = 2$

$-9 \gg 2 = -3$

Операция правого сдвига без расширения знака (>>>).

Операция сдвигает биты первого операнда вправо на число позиций, указанных во втором операнде. Значения выдвинутых битов пропадают, а на место сдвинутых битов слева записываются нули.

Например,

$19 \ggg 2 = 4$.

1.3 Операторы присваивания

К операторам присваивания относят следующие:

=, +=, -=, *=, /=

Операторы присваивания назначают левому операнду значение правого операнда. Основным оператором присваивания – равенство (=). Таким образом, оператор $x = u$ присваивает переменной x значение переменной u .

Сокращенная форма стандартных арифметических операций следующая:

$x += y$ означает $x = x + y$;
 $x -= y$ означает $x = x - y$;
 $x *= y$ означает $x = x * y$;
 $x /= y$ означает $x = x / y$;
 $x \% = y$ означает $x = x \% y$.

Имеются дополнительные операторы присваивания для битовых операций:

$x << y$ означает $x = x << y$;
 $x >> y$ означает $x = x >> y$;
 $x >>> y$ означает $x = x >>> y$;
 $x \& y$ означает $x = x \& y$;
 $x | y$ означает $x = x | y$.

1.4 Логические операторы

Логические операции воспринимают операнды как логические значения true или false. Результатом также будет логическое значение.

Оператор And (&&).

Формат:

expr1 && expr2

Логическая операция “and” возвращает значение true, если оба логических выражения *expr1* и *expr2* есть true, иначе – false.

Оператор Or (||).

Формат:

expr1 || expr2

Логическая операция “or” возвращает значение true, если хотя бы одно из логических выражений *expr1* или *expr2* есть true. Если оба выражения есть false, то и результатом будет false.

Оператор Not (!).

Формат:

!expr

Логическая операция “not” – унарный оператор, который дает отрицание его операнду. Таким образом, если значение *expr* есть true результатом будет false, а если значение *expr* есть false результатом будет true.

Примеры:

- 1) `if(a<10) && (c>=20))`
`{...}`
- 2) `if(!confirm(“Are you sure?”))`
`{...}`

Операции сравнения (==, >, >=, <, <=, !=).

Операции сравнивают оба своих операнда, и результатом сравнения будет логическое значение true или false. Операндами могут быть числовые или строковые значения.

Обозначения операций сравнения следующее:

- равно (==): возвращаемое значение будет true, если операнды равны;
- не равно (!=): возвращаемое значение будет true, если операнды не равны;
- больше (>): возвращаемое значение будет true, если левый операнд больше правого;
- больше или равно (>=): возвращаемое значение будет true, если левый операнд больше правого или равен ему;
- меньше (<): возвращаемое значение будет true, если левый операнд меньше правого;
- Меньше или равно (<=): возвращаемое значение будет true, если левый операнд меньше правого или равен ему.

Пример:

```
If (age <= 18) || (height < 150)
```

```
Alert ("You are too young!")
```

1.5 Строковые операторы

В дополнение к операциям сравнения к строковым значениям применяется операция конкатенации (+), выполняющая слияние двух строк в одну. Возвращаемым значением операции будет новая строка.

Например, операция

“my ”+”string” возвращает значение “my string”.

Существует укороченная форма операции конкатенации (+=).

Например, если переменная mystring содержит строку “alpha”, то выражение mystring += “bet” присвоит переменной mystring значение “alphabet”.

1.6 Условные операторы

Реализация разветвляющейся базовой структуры в языке сценариев JavaScript осуществляется с помощью трехместного оператора проверки условия, оператора ветвления и оператора switch.

Оператор проверки условия (трехместный).

(condition)? Val1:val2

Если условие принимает значение true, выражение получает значение val, в противном случае – значение val2.

Например:

```
Status = (age >= 18)? “adult” : “minor”.
```

Оператор ветвления if...else.

```
If (условие) {
операторы}
[ else {
операторы}]
```

Условием может быть любое JavaScript выражение, вычисляющее логические значения true или false, включая вложенные if операторы. Множественное задание операторов должно заключаться в фигурные скобки.

Пример:

```
function checkDate () {
  if (document.form1.threeChar.value.length == 3) {
    return true;
  } else {
    alert ("Enter exactly three characters." + document.form1.threeChar.value + "is not valid.")
    return false;
  }
}
```

Оператор switch.

```
switch (выражение) {
  case знач1:
    [операторы1]
    break;
  case знач2:
    [операторы2]
    break;
  ...
  default:
    [операторы]
}
```

Количество секций не ограничено

Вычисляется значение выражения и оно последовательно сравнивается со всеми знач1, знач2 и так далее. В случае совпадения выполняется соответствующая группа операторов. Если нет совпадения ни с одним значением, то выполняются операторы default, в случае если эта группа операторов не задана, происходит выход из оператора switch.

Все оставшиеся операторы JavaScript можно объединить в следующие группы:

операторы цикла:

```
for
while
do...while
```

```
break
continue
```

операторы для работы с объектами:

```
for...in
new
this
with
```

операторы комментария:

```
//
/* ... */
```

1.7 Операторы цикла

Цикл — группа команд, выполняющаяся многократно в зависимости от условия повторения. В JavaScript поддерживаются операторы цикла for и while (начиная с версии JavaScript 1.2 поддерживается также do...while), а также вспомогательные операторы break и continue, выполняющиеся внутри цикла. Оператор цикла for...in применяется для объектов.

Оператор for.

Цикл типа for выполняется до тех пор пока заданное условие не примет значение false.

Формат оператора:

```
for ([initial-expression];[condition];[increment-expression]) {
  statements
}
```

В начале цикла вычисляются операторы инициализации цикла increment-expression. Операторы тела цикла statements выполняются до тех пор, пока условие condition имеет значение true. Выражение increment-expression выполняется при каждом повторе цикла.

Последовательность выполнения цикла:

- 1) инициализация цикла — выполняется выражение increment-expression, если оно имеется. Выражение обычно содержит несколько счетчиков или переменных цикла;
- 2) вычисляется условное выражение condition. Если его значение равно false, цикл прекращается;
- 3) выполняется выражение increment-expression;
- 4) выполняются операторы цикла statements и управление передается на шаг 2.

Оператор while.

Оператор цикла while выполняет повторение до тех пор, пока значение заданного условия остается равным true.

Формат оператора:

```
while (condition) {
  statements
}
```

Проверка условия condition выполняется после завершения работы операторов цикла statements перед переходом на повторение цикла. Проверка цикла не непрерывна, и выполняется один раз при первом вхождении в цикл и повторно при окончании выполнения операторов цикла.

Пример:

```
n=0
x=0
while (n<3) {
  n++
  x+=n
}
```

После окончания цикла значения переменных: n=3, x=6.

Пример (бесконечный цикл):

```
while (true) {
  alert ("Hello, world")
}
```

Оператор break.

Оператор break прекращает выполнение циклов while или for и передает управление оператору программы, следующему за циклом.

Пример:

Цикл заканчивается при значении переменной i, равном 3 и возвращается значение функции 3*x.

```
function testBreak(x) {
  var i = 0
  while (i<6) {
    if (i == 3)
      break
    i++
  }
  return i*x
}
```

Оператор continue.

Оператор continue прекращает выполнение блока операторов цикла while или for и продолжение цикла начинается с новой итерации, а именно:

- в while цикле происходит передача управления на проверку условия condition;
- в for цикле передается управление на вычисление выражения increment-expression.

Пример:

Переменная n принимает значения 1, 3, 7 и 12.

```
i = 0
n = 0
while (i<5) {
  i++
  if (i==3)
    continue
  n+=i
}
```

Оператор do...while.

```
do {
  операторы
}
while (условие);
```

Этот цикл гарантирует, что операторы будут выполняться хотя бы один раз независимо от того, выполняется ли условие.

1.8 Операторы для работы с объектами

В JavaScript имеется несколько способов манипулирования объектами, а именно:

- оператор цикла for...in;
- оператор new;
- оператор with;
- оператор this.

Оператор for...in.

Оператор for...in выполняет итерацию по всем свойствам объекта. Для каждого имеющегося свойства объекта выполняются заданные операторы цикла.

Формат оператора:

```
for (variable in object) {
  statements }
```

Пример:

Приведенная функция получает в качестве аргумента объект и его имя. В функции перебираются все свойства объекта и в результате формируется текстовая строка с перечислением свойств объекта, их значений и управляющих HTMLтегов.

```
functiondump_props (obj,obj_name) {var result=""
for (vari in obj) {
result +=obj_name+"."+i+" = "+obj[i]+"<BR>"
}
result += "<HR>"
return result
}
```

Оператор new.

Оператор new позволяет создавать определенные пользователем объекты.

Форматоператора:

```
objectName = new objectType (param1 [,param2] ...[,paramN])
```

Ключевое слово this.

Ключевое слово this используется при обращении к текущему объекту. В общем случае, с помощью this происходит обращение к вызываемому объекту в методах.

Формат:

```
this[.propertyName]
```

Пример:

Функция validate определяет допустимость свойств объекта, заданных нижними и верхними пределами их значений.

```
function validate (obj, lowval, hival)
if ((obj.value<lowval) || (obj.value>hival))
alert("InvalidValue!")
}
```

Можно вызвать функцию из обработчика событий onChange, используя this для передачи объекта как параметра:

```
<b> Enter a number between 18 and 99:</b>
<input type="text" NAME="age" SIZE=3 onChange="validate(this, 18,99)">
```

Листинг скрипт-программы с использованием ключевого слова this представлен ниже.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document this</title>
</script>
function validate (obj,lowval,hival){
if ((obj.value<lowval) || (obj.value>hival))
alert("Invalid Value!")
}
</script>

</head>
<body>
<b> Enter a number between 18 and 99:</b>
<input type=text NAME=age SIZE=3 onChange=validate(this,18,99)>
</body>
</html>
```

Оператор with.

Оператор with используется для установки значения умолчания типа объекта для блока операторов. Внутри этого блока можно не указывать имени объекта при обращении к его свойствам.

Формат оператора:

```
With (object) {
  Statements
}
```

Пример:

```
var a,x,y
var r=1
with (Math) {
  a=PI*r*r
  x=r*cos(PI)
  y=r*sin(PI/2)
}
```

1.9 Комментарии

Комментарии являются пояснениями автора всего скрипт-программе. Комментарии игнорируются интерпретаторами.

Комментарии в одной строке, заданные двумя слэшами (//);

Комментарии группы строк задаются как: /*...*/.

Пример:

```
// This is single-line comment.
/* This is a multiple-line comment. It can be of any length, and you can put whatever you want
here.*/
```

2 Старшинство операций

Старшинство операций заключается в приоритете выполнения действий в сложных выражениях. Возможно, используя скобки, изменить последовательность выполнения операций.

Старшинство операций в порядке возрастания приоритетов следующее:

запятая ,

присвоение = += -= *= /= %= <<= >>= >>>= &= ^= ||=

проверка условия ?

логическое or ||

логическое and &&

двоичное or ||

двоичное xor ^

двоичное and &

сравнение == !=

соотношение <<= >>=

двоичный сдвиг <<>>>>

сложение/вычитание + -

умножение/деление * / %

отрицание/инкрементация ! ++ --

Если не уверены в приоритете операции, лучше расставить скобки.